UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

13 – Querying & Indexing for Performance Optimization in MongoDB

DATABASE SYSTEM

Vincentius Kurniawan S.Kom., M.Eng.Sc.,

COURSE OBJECTIVES

GOALS.

1 Explain and Implement query in MongoDB

2 Implement query in Embedded documents

3 Explain and Implement Indexing

4 Deciding which type of indexing to use

5 Implement aggregation and complex query

OUTLINES.

- Structure of a query (`find()`, projections, filters)
- Common operators (`$eq`, `$gt`, `$in`, `$regex`)
- Querying in Embedded
- Indexing
- Type of Indexes
- Aggregation and Pipelines

QUERYING

INTRODUCTION TO FIND

The `find` method is used to perform queries in MongoDB. Querying returns a subset of documents in a collection, from no documents at all to the entire collection. Which documents get returned is determined by the first argument to find, which is a document specifying the query criteria.

```
db.collection.find(<filter>, <projection>)

//- <filter>: Specifies the conditions documents must meet.
//- <projection>: Specifies which fields to include or exclude from the result
```

```
//An empty query document (i.e., {}) matches everything in the collection. If find isn't given a query document, it defaults to {}.
db.products.find()

//adding key/value pairs to the query document, to restrict the search
db.products.find({ category: "Electronics" })

//Multiple conditions which gets interpreted as "condition1 AND condition2 AND …AND conditionN."
db.products.find({ category: "Electronics", name:"Laptop" })

//a second argument to find (or findOne) specifying the keys you want.
db.products.find({ category: "Electronics" }, { name: 1, price: 1, _id: 0 })
```

QUERYING

QUERY CONDITIONAL

`"$lt"`, `"$lte"`, `"$gt"`, and `"$gte"` are all comparison operators, corresponding to <, <=, >, and >=, respectively. They can be combined to look for a range of values

To query for documents where a key's value is not equal to a certain value, you must use another conditional operator, `"$ne"`, which stands for "not equal."

```
//This would find all documents where the "age" field was
greater than or equal to 18 AND less than or equal to 30.
> db.users.find({"age" : {"$gte" : 18, "$lte" : 30}})

//to find people who registered before January 1, 2007,
> start = new Date("01/01/2007")
> db.users.find({"registered" : {"$lt" : start}})

//find all users who do not have the username "joe"
> db.users.find({"username" : {"$ne" : "joe"}})
```

QUERYING

“OR” QUERY

There are two ways to do an OR query in MongoDB. `"$in"` can be used to query for a variety of values for a single key. `"$or"` is more general; it can be used to query for any of the given values across multiple keys.

The opposite of `"$in"` is `"$nin"`, which returns documents that don’t match any of the criteria in the array

```
//For instance, suppose we're running a raffle and the winning ticket
numbers are 725, 542, and 390.
> db.raffle.find({"ticket_no" : {"$in" : [725, 542, 390]}})

//For example, if we are gradually migrating our schema to use
usernames instead of user ID numbers, we can query for either by
using this. This matches documents with a "user_id" equal to 12345
and documents with a "user_id" equal to "joe".
> db.users.find({"user_id" : {"$in" : [12345, "joe"]}})

//This query returns everyone who did not have tickets with those
numbers.
> db.raffle.find({"ticket_no" : {"$nin" : [725, 542, 390]}})

//To find documents where "ticket_no" is 725 or "winner" is true
> db.raffle.find({"$or" : [{"ticket_no" : 725}, {"winner" : true}]})
```

QUERYING

“NOT” QUERY

"$not" is a metaconditional: it can be applied on top of any other criteria. "$not" can be particularly useful in conjunction with regular expressions to find all documents that don’t match a given pattern

```
//As an example, let’s consider the modulus operator, "$mod".
"$mod" queries for keys whosevalues, when divided by the first
value given, have a remainder of the second value. returns users
with "id_num"s of 1, 6, 11, 16, and so on
> db.users.find({"id_num" : {"$mod" : [5, 1]}})

//If we want, instead, to return users with "id_num"s of 2, 3, 4,
5, 7, 8, 9, 10, 12, etc., we can use "$not":
> db.users.find({"id_num" : {"$not" : {"$mod" : [5, 1]}}})
```

QUERYING

“NULL” TYPE-SPECIFIC QUERY

`null` behaves a bit strangely. It does match itself, so if we have a collection with the following documents:

```
> db.c.find()
{ "_id" : ObjectId("4ba0f0dfd22aa494fd523621"), "y" : null }
{ "_id" : ObjectId("4ba0f0dfd22aa494fd523622"), "y" : 1 }
{ "_id" : ObjectId("4ba0f148d22aa494fd523623"), "y" : 2 }
```

we can query for documents whose "y" key is null in the expected way:

```
> db.c.find({"y" : null})
{ "_id" : ObjectId("4ba0f0dfd22aa494fd523621"), "y" : null }
```

However, null also matches “does not exist.” Thus, querying for a key with the value null will return all documents lacking that key:

```
> db.c.find({"z" : null})
{ "_id" : ObjectId("4ba0f0dfd22aa494fd523621"), "y" : null }
{ "_id" : ObjectId("4ba0f0dfd22aa494fd523622"), "y" : 1 }
{ "_id" : ObjectId("4ba0f148d22aa494fd523623"), "y" : 2 }
```

If we only want to find keys whose value is null, we can check that the key is null and exists using the "$exists" conditional:

```
> db.c.find({"z" : {"$eq" : null, "$exists" : true}})
```

QUERYING

“REGEX” TYPE-SPECIFIC QUERY

"$regex" provides regular expression capabilities for pattern matching strings in queries. Regular expressions are useful for flexible string matching. Regular expression flags (e.g., i) are allowed but not required

MongoDB uses the Perl Compatible Regular Expression (PCRE) library to match regular expressions; any regular expression syntax allowed by PCRE is allowed in MongoDB.

```
//For example, if we  want to find all users with the
name “Joe” or “joe,” we can use a regular expression to
do case-insensitive matching:
> db.users.find( {"name" : {"$regex" : /joe/i } })

//If we want to match not only various capitalizations
of “joe,” but also “joey,”
> db.users.find({"name" : /joey?/i})
```

QUERYING

“ARRAY” TYPE-SPECIFIC QUERY

Querying for elements of an array is designed to behave the way querying for scalars does. For example, if the array is a list of fruits, like this:

```
> db.food.insertOne({"fruit" : ["apple", "banana", "peach"]})
```

the following query will successfully match the document:

```
> db.food.find({"fruit" : "banana"})
```

We can query for it in much the same way as we would if we had a document that looked like the (illegal) document `{"fruit" : "apple", "fruit" : "banana","fruit" : "peach"}`.

Others operator that can be used to query an array:

- `$all`
- `$size`
- `$slice`

```
//find all documents with both "apple" and "banana"
> db.food.find({fruit : {$all : ["apple", "banana"]}})
{"_id" : 1, "fruit" : ["apple", "banana", "peach"]}
{"_id" : 3, "fruit" : ["cherry", "banana", "apple"]}

//If you want to query for a specific element of an array, you can
specify an index using the syntax key.index:
> db.food.find({"fruit.2" : "peach"})

//query for arrays of a given size
> db.food.find({"fruit" : {"$size" : 3}})

// For example, suppose we had a blog post document and we wanted
to return the first 10 comments:
> db.blog.posts.findOne(criteria, {"comments" : {"$slice" : 10}})
```

QUERYING ON EMBEDDED DOCUMENTS

There are two ways of querying for an embedded document: querying for the whole document or querying for its individual key/value pairs. Querying for an entire embedded document works identically to a normal query

```
//For example, if we have a document that looks like this:
{
  "name" : {
    "first" : "Joe",
    "last" : "Schmoe"
  },
  "age" : 45
}
```

we can query for someone named Joe Schmoe with the following:

```
> db.people.find({"name" : {"first" : "Joe", "last" : "Schmoe"}
```

Problem: a query for a full subdocument must exactly match the subdocument. If Joe decides to add a middle name field, suddenly this query won't work anymore; it doesn't match the entire embedded document! This type of query is also order sensitive: `{"last" : "Schmoe", "first" : "Joe"}` would not be a match.

QUERYING

QUERYING ON EMBEDDED DOCUMENTS (CONT.D)

Solution: Query for just a specific key or keys of an embedded document. Then, if your schema changes, all of your queries won't suddenly break because they're no longer exact matches. You can query for embedded keys using dot notation:

```
> db.people.find({"name.first" : "Joe", "name.last" : "Schmoe"})
```

This dot notation is the main difference between query documents and other document types. Query documents can contain dots, which mean "reach into an embedded document." Dot notation is also the reason that documents to be inserted cannot contain the . character.

QUERYING ON EMBEDDED DOCUMENTS (USING ELEMENT MATCH)

"`$elemMatch`" allows you to “group” and to partially specify criteria to match a single embedded document in an array

```
> db.blog.find()
{
  "content" : "...",
  "comments" : [
    {
    "author" : "joe",
    "score" : 3,
    "comment" : "nice post"
    },
    {
    "author" : "mary",
    "score" : 6,
    "comment" : "terrible post"
    }
  ]
}

//Example that will now work
//db.blog.find({
//  "comments" : {
//    "author" : "joe",
//    "score" : {"$gte" : 5}
//  }
//})
//OR
//db.blog.find({
//  "comments.author" : "joe",
//  "comments.score" : {
//    "$gte" :5
//  }
//}),


// Correct Query
> db.blog.find({"comments" : {"$elemMatch" : {
  "author" : "joe",
  "score" : {"$gte" : 5}
}}})
```

QUERYING

“WHERE” QUERIES

"`$elemMatch`" allows you to “group” and to partially specify criteria to match a single embedded document in an array

```
> db.blog.find()
{
  "content" : "...",
  "comments" : [
    {
    "author" : "joe",
    "score" : 3,
    "comment" : "nice post"
    },
    {
    "author" : "mary",
    "score" : 6,
    "comment" : "terrible post"
    }
  ]
}

//Example that will now work
//db.blog.find({
//  "comments" : {
//    "author" : "joe",
//    "score" : {"$gte" : 5}
//  }
//})
//OR
//db.blog.find({
//  "comments.author" : "joe",
//  "comments.score" : {
//    "$gte" :5
//  }
//}),


// Correct Query
> db.blog.find({"comments" : {"$elemMatch" : {
  "author" : "joe",
  "score" : {"$gte" : 5}
}}})
```

QUERYING

“WHERE” QUERIES

"`$elemMatch`" allows you to “group” and to partially specify criteria to match a single embedded document in an array

```
> db.blog.find()
{
  "content" : "...",
  "comments" : [
    {
    "author" : "joe",
    "score" : 3,
    "comment" : "nice post"
    },
    {
    "author" : "mary",
    "score" : 6,
    "comment" : "terrible post"
    }
  ]
}

//Example that will now work
//db.blog.find({
//  "comments" : {
//    "author" : "joe",
//    "score" : {"$gte" : 5}
//  }
//})
//OR
//db.blog.find({
//  "comments.author" : "joe",
//  "comments.score" : {
//    "$gte" :5
//  }
//}),


// Correct Query
> db.blog.find({"comments" : {"$elemMatch" : {
  "author" : "joe",
  "score" : {"$gte" : 5}
}}})
```

INTRODUCTION TO INDEXING

By default, queries that lack appropriate indexes result in a collection scan, where MongoDB examines each document individually to determine whether it satisfies the query criteria—a method that becomes increasingly inefficient as the dataset grows.

To address this, MongoDB supports indexes built on a B-tree data structure, which organizes indexed fields in a sorted manner. This allows the query engine to locate matching documents more efficiently, often avoiding the need to examine the entire collection.

Conceptually, an index functions much like the index section of a textbook: instead of reading every page, the reader can jump directly to the relevant content using a reference.

```
//Create a Single-field Index
db.users.createIndex({ username: 1 })  // Ascending order

//Query Using the index
db.users.find({ username: "raphael" })

//Analyze Index Usage
db.users.find({ username: "raphael" }).explain("executionStats")
```

INDEXING

INDEX CARDINALITY

Cardinality refers to how many distinct values there are for a field in a collection. It helps determine how selective a query can be

Type of Cardinality:

Type	Description	Examples
Low	Few distinct values	Gender, Opt-out Flag (true/false)
Medium	Moderate variation	Age, Zip Code
High	Many unique values	Email, Username, ID

Cardinality is key for effective indexing: high-cardinality fields (many unique values) allow indexes to significantly narrow search results, as each index entry points to fewer documents.

Meanwhile, low-cardinality fields (few unique values) provide less filtering, often forcing the database to scan more documents, thus reducing the index's performance benefit.

```
// Compound index example (good): In compound indexes, place
high-cardinality fields first to improve selectivity during
filtering
db.users.createIndex({ "name": 1, "gender": 1 })

// Less ideal if reversed:
db.users.createIndex({ "gender": 1, "name": 1 })

//Prioritize high-cardinality fields for indexing to maximize
performance
```

INDEXING

WHEN NOT TO INDEX

If the query returns a large percentage of the collection, indexes may slow it down. A rule of thumb: indexes help when querying <30% of the data. But this range can vary from 2% to 60%, depending on: Collection size, Document size, and Query selectivity.

```
> db.entries.find({"created_at" : {"$lt" : hourAgo}})

//We index "created_at" to speed up this query.
//When we first launch, the result set is tiny and the query
returns instantly. But after a couple of weeks, it starts being
a lot of data, and after a month this query is already taking
too long to run.
```

Indexes Work Well For	Collection Scans Work Well For
Large collections	Small collections
Large documents	Small documents
Selective queries	Nonselective queries

INDEXING

INDEX TYPE

MongoDB offers a variety of index types, each created for different query patterns and data structures

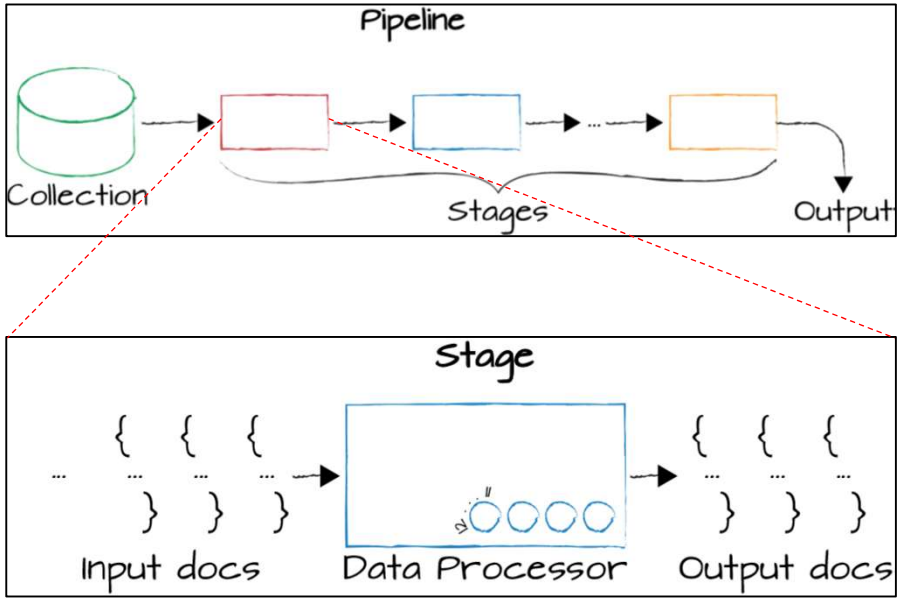
Index Type	Purpose
Single Field	Indexes one field for fast equality or range queries
Compound	Indexes multiple fields; order matters for query optimization
Multikey	Automatically created when indexing arrays; supports queries on array items
Text	Enables full-text search on string fields
Hashed	Uses hashed values for even data distribution (e.g., in sharded clusters)
Wildcard	Indexes dynamic or unknown fields using a flexible pattern
2D	Legacy coordinate pairs on a flat plane
2DSphere	GeoJSON objects and spherical geometry queries

AGGREGATION AND PIPELINE

Aggregation is MongoDB's framework for performing advanced data analysis and transformation directly within the database. It allows you to filter documents, group and summarize data, sort, reshape, and compute values, join collection and build reports

Think of it as MongoDB's version of SQL's `GROUP BY`, `JOIN`, and `HAVING`.

The aggregation framework is based on the concept of a pipeline. With an aggregation pipeline we take input from a MongoDB collection and pass the documents from that collection through one or more stages, each of which performs a different operation on its inputs



AGGREGATION

COMMON STAGES OPERATIONS

Operation	Purpose	Example Usage
`$match`	Filters documents (like SQL `WHERE`)	`{ $match: { age: { $gte: 30 } } }`
`$group`	Groups documents and computes values	`{ $group: { _id: "$city", count: { $sum: 1 }}}`
`$sort`	Sorts documents	`{ $sort: { created_at: -1 } }`
`$project`	Reshapes documents (select fields)	`{ $project: { name: 1, age: 1, _id: 0 } }`
`$limit`	Limits number of documents	`{ $limit: 10 }`
`$lookup`	Joins with another collection	`{ $lookup: { from: "orders", localField: "userId", foreignField: "userId", as: "orders" }}`

AGGREGATION

BASIC STRUCTURE

Example: Filtering and Calculate Total Sales for each Region:

```
db.collection.aggregate([
  // Stage 1: Filter
  { $match: { status: "active" } },
  // Stage 2: Group
  { $group: { _id: "$region", total: { $sum: "$sales" } } },
  // Stage 3: Sort
  { $sort: { total: -1 } }
])
```

Example: Calculate Average Age by City:

```
db.users.aggregate([
  { $group: { _id: "$city", avgAge: { $avg: "$age" } } }
])
```

JOINING COLLECTION

Collection 1 User:

```
{ "_id": 1, "name": "Alice", "email": "alice@example.com" }
{ "_id": 2, "name": "Bob", "email": "bob@example.com"
```

Collection 2 Orders:

```
{ "_id": 101, "userId": 1, "item": "Laptop", "price": 1200 }
{ "_id": 102, "userId": 1, "item": "Mouse", "price": 25 }
{ "_id": 103, "userId": 2, "item": "Keyboard", "price": 75 }
```

Aggregation Pipeline using `$lookup`:

```
db.users.aggregate([
  {
    $lookup: {
      from: "orders",           // Collection to join
      localField: "_id",        // Field from 'users'
      foreignField: "userId",   // Field from 'orders'
      as: "userOrders"          // Output array field
    }
  }
])
```

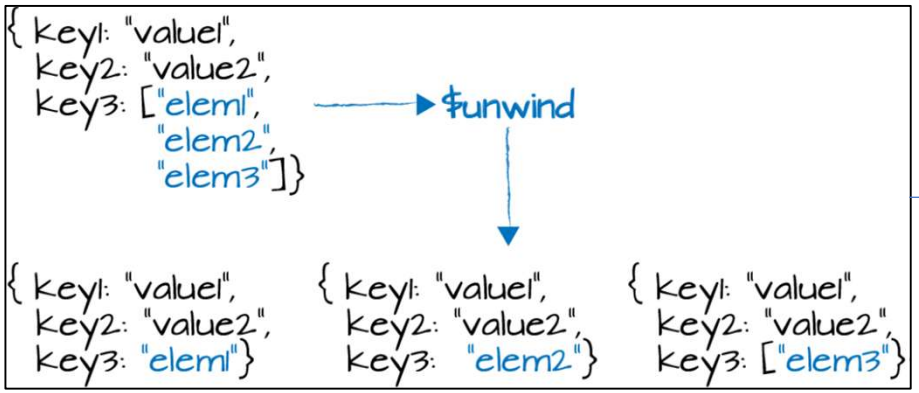
Output:

```
{
  "_id": 1,
  "name": "Alice",
  "email": "alice@example.com",
  "userOrders": [
    { "_id": 101, "userId": 1, "item": "Laptop", "price": 1200 },
    { "_id": 102, "userId": 1, "item": "Mouse", "price": 25 }
  ]
}
{
  "_id": 2,
  "name": "Bob",
  "email": "bob@example.com",
  "userOrders": [
    { "_id": 103, "userId": 2, "item": "Keyboard", "price": 75 }
  ]
}
```

AGGREGATION

UNWIND OPERATION

When working with array fields in an aggregation pipeline, it is often necessary to include one or more unwind stages. This allows us to produce output such that there is one output document for each element in a specified array field.



Explanation: In this example, we have an input document that has three keys and their corresponding values. The third key has as its value an array with three elements. `$unwind` if run on this type of input document and configured to unwind the `key3` field will produce documents that look like those shown at the bottom of image

AGGREGATION

UNWIND OPERATION: BASIC USAGE

Collection 1 Employees:

```
[
  { "_id": 1, "name": "Alice", "skills": ["Python", "MongoDB", "Docker"] },
  { "_id": 2, "name": "Bob", "skills": ["Java", "Kubernetes"] },
  { "_id": 3, "name": "Charlie", "skills": [] }
]
```

Basic `$unwind` usage:

```
db.employees.aggregate([
  { $unwind: "$skills" }
])
```

Results:

```
{ "_id": 1, "name": "Alice", "skills": "Python" }
{ "_id": 1, "name": "Alice", "skills": "MongoDB" }
{ "_id": 1, "name": "Alice", "skills": "Docker" }
{ "_id": 2, "name": "Bob", "skills": "Java" }
{ "_id": 2, "name": "Bob", "skills": "Kubernetes" }
```

Notice: Charlie's document is excluded because his skills array is empty.

UNWIND OPERATION: PRESERVE EMPTY ARRAY

Collection 1 Employees:

```
[
  { "_id": 1, "name": "Alice", "skills": ["Python", "MongoDB", "Docker"] },
  { "_id": 2, "name": "Bob", "skills": ["Java", "Kubernetes"] },
  { "_id": 3, "name": "Charlie", "skills": [] }
]
```

To include documents with empty or missing arrays:

```
db.employees.aggregate([
  {
    $unwind: {
      path: "$skills",
      preserveNullAndEmptyArrays: true
    }
  }
])
```

Results:

```
{ "_id": 1, "name": "Alice", "skills": "Python" }
{ "_id": 1, "name": "Alice", "skills": "MongoDB" }
{ "_id": 1, "name": "Alice", "skills": "Docker" }
{ "_id": 2, "name": "Bob", "skills": "Java" }
{ "_id": 2, "name": "Bob", "skills": "Kubernetes" }
{ "_id": 3, "name": "Charlie", "skills": null }
```

AGGREGATION

UNWIND OPERATION: INCLUDE ARRAY INDEX

Collection 1 Employees:

```
[
  { "_id": 1, "name": "Alice", "skills": ["Python", "MongoDB", "Docker"] },
  { "_id": 2, "name": "Bob", "skills": ["Java", "Kubernetes"] },
  { "_id": 3, "name": "Charlie", "skills": [] }
]
```

To track the position of each skill:

```
db.employees.aggregate([
  {
    $unwind: {
      path: "$skills",
      includeArrayIndex: "skillIndex",
    }
  }
])
```

Results:

```
{ "_id": 1, "name": "Alice", "skills": "Python", "skillIndex": 0 }
{ "_id": 1, "name": "Alice", "skills": "MongoDB", "skillIndex": 1 }
{ "_id": 1, "name": "Alice", "skills": "Docker", "skillIndex": 2 }
{ "_id": 2, "name": "Bob", "skills": "Java", "skillIndex": 0 }
{ "_id": 2, "name": "Bob", "skills": "Kubernetes", "skillIndex": 1 }
```

SUMMARY

Query Structure
A MongoDB query typically starts with `find()` to retrieve documents, accompanied by filters and projections to refine and shape the results.

Common Operators
Operators like `$eq`, `$gt`, `$in`, and `$regex` empower expressive filtering logic by targeting specific values, ranges, and patterns.

Embedded Document Queries
Accessing nested fields requires dot notation or operator combinations to accurately filter within subdocuments and arrays

Indexing
Indexes drastically improve read performance by allowing MongoDB to locate matching documents faster than a full collection scan.

Types of Indexes
MongoDB supports diverse index types including single field, compound, multikey, text, and hashed — each optimized for different query patterns.

Aggregation & Pipeline
The aggregation framework enables powerful data transformations via staged operators like `$match`, `$group`, `$project`, and `$lookup`

REFERENCES

Elmasri, Ramez and Shamkant B. Navathe (2016), Fundamentals of Database Systems, 7th edition, Addison Wesley

Van der Lans, R. F. (2007). SQL for MySQL developers. Addison Wesley.

Swan, M. (2015). Blockchain: Blueprint for a new economy. O'Reilly Media.

Bradshaw, D., Brazil, K., & Chodorow, K. (2019). MongoDB: The definitive guide—Powerful and scalable data storage. O'Reilly Media.

Copeland, R. (2013). MongoDB applied design patterns. O'Reilly Media.

NEXT OUTLINES

NEXT WEEK

- Presentation Final Project

KEEP IN TOUCH

THANK YOU

vincentius.kurniawan@lecturer.umn.ac.id

@aixeta#2683